Vision-Guided Pick and Place: How the Code Works

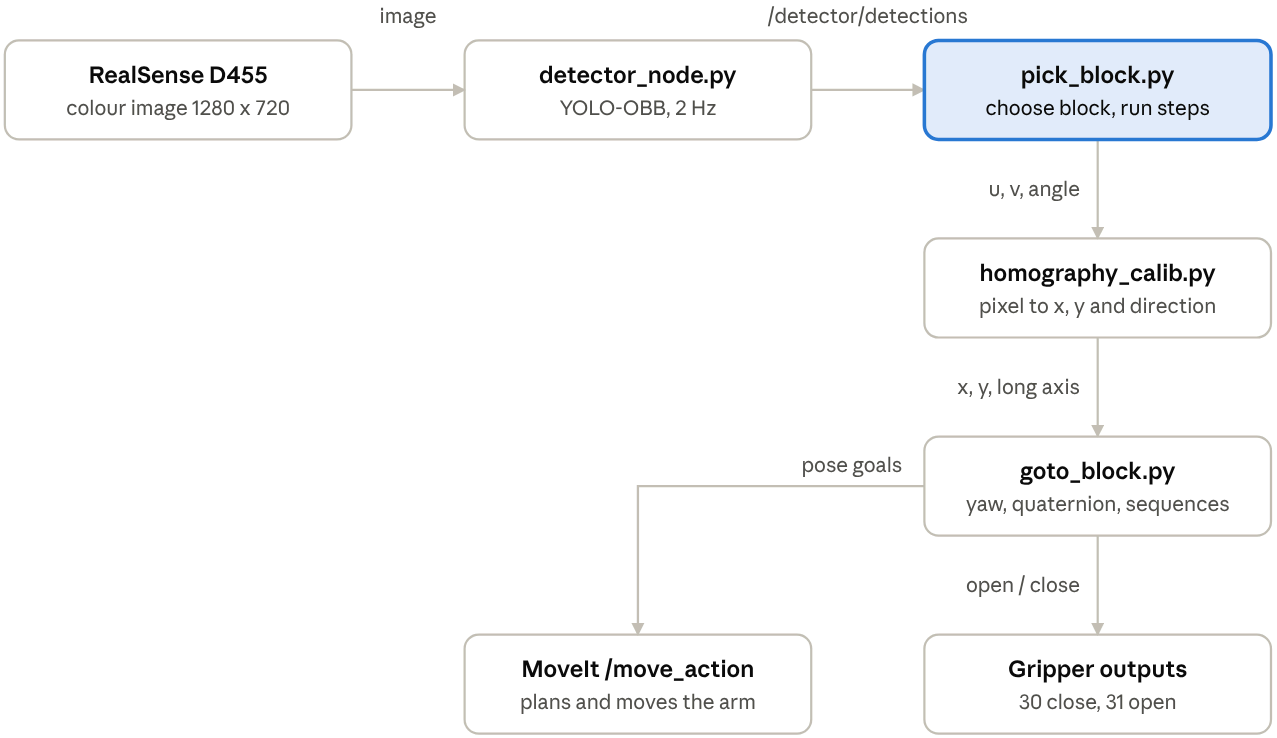
Oct 30, 2026 · @karthik

The overhead camera finds a block, a homography turns its pixel into robot x and y, the block angle becomes a link6 yaw, and MoveIt plus two digital outputs pick and place it. This doc explains every step of homography_calib.py, detector_node.py, goto_block.py and pick_block.py, with the maths behind them.

1. Big picture

Four files split the job: one sees, one converts, two move. Data flows one way, from the camera image to the robot, and only two numbers per block cross between them: a pixel and an angle.

Two numbers per block cross from camera to robot: a pixel and an angle



data flow · 7 parts, from the camera image to the arm and the gripper

The detector never talks to the robot; pick_block.py reads one message, and everything after it is maths and motion.

File	Job	Runs as
detector_node.py	YOLO-OBB on the colour image; publishes each block's pixel centre (u, v) and long-side angle in the image	ROS 2 node, 2 Hz, with a live window
homography_calib.py	Holds the calibration and the maths: pixel to table x, y, height correction, image angle to robot direction	Imported module (also a command-line tool for query, fit, selftest)
goto_block.py	Gripper yaw, quaternion, the MoveIt move, the gripper outputs, the pick and place sequences	Imported by pick_block.py; also a test tool for one pixel
pick_block.py	Reads one detection message, chooses the block, runs pick and optionally place	Script you start for each pick

Nothing in this chain uses depth or a camera TF. Position comes only from the homography and the known heights of the table, board and block.

2. Coordinate frames

Four frames appear in the code, and every number in it belongs to one of them. Getting the frame right is most of the work.

Frame	Origin	Axes	Used for
base_link	Robot base	x forward, y left, z up (right-handed)	Every target position and yaw the robot is sent

Frame	Origin	Axes	Used for
link6	Robot flange	Pointing down at a pick: its z = -z of base_link, its x lies flat on the table at angle yaw	What MoveIt moves; tf2_echo base_link link6 reads it
camera_color_optical_frame	Colour lens of the D455	x right in the image, y down in the image, z out of the lens	Only for the optional camera pose; not used by the pick code
Image pixels (u, v)	Top-left pixel	u to the right, v down, 1280 x 720	Everything the detector outputs

Where things are, in base_link. The table top is at z = -0.042 m. The camera lens is at x 0.294, y -0.019, z 0.721 m, which is 0.762 m above the table. It looks almost straight down (about 1.2 deg off). The table point right under the camera (the nadir) is x 0.294, y -0.019 m, and it shows up at pixel (642, 376), close to the image centre.

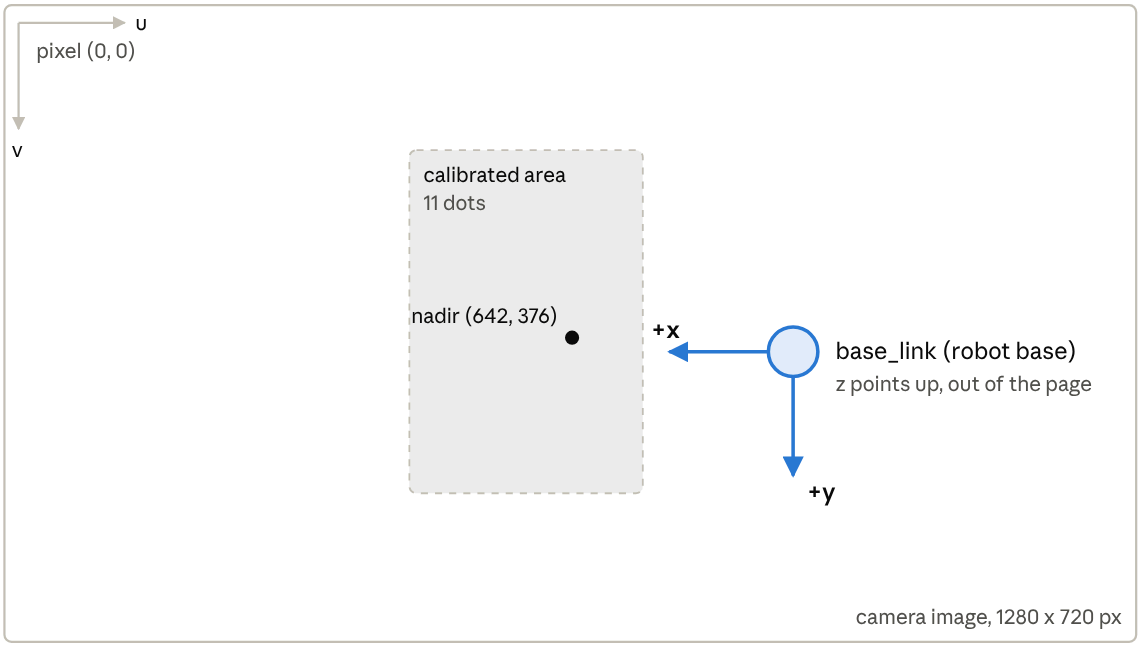
How the image axes lie on the table. Near the image centre, one pixel is about 1.18 mm on the table:

- 1 px to the right (+u) moves -1.175 mm in x and -0.088 mm in y, so image right is about robot -x.
- 1 px down (+v) moves -0.083 mm in x and +1.185 mm in y, so image down is about robot +y.

The image is a mirror view of the table. Seen from above, base_link turns counter-clockwise from x to y. On the screen, u to v turns clockwise. So an angle measured in the image runs the other way on the table. Section 6 shows how the code handles this.

In the camera image, robot x points left and robot y points down

Top view as the camera sees it, half scale; 1 px is about 1.18 mm on the table



1 px right (+u): -1.175 mm in x, -0.088 mm in y, so image right is about robot -x

1 px down (+v): -0.083 mm in x, +1.185 mm in y, so image down is about robot +y

camera view of the table · positions from the homography; robot base at pixel (892, 392)

The base_link origin, pixel about (892, 392), sits under the robot arm, so it is never visible itself; its axes are drawn where the homography puts them.

3. Calibration: pixel to table with a homography

The calibration is one 3 x 3 matrix H that turns any pixel into a point on the table, accurate to about 1.0 mm on points it was not fitted to. It works because the camera is fixed and everything you pick lies on one flat plane.

Why a plane needs only a 3 x 3 matrix

A pinhole camera maps a 3D point (X, Y, Z) to a pixel like this, with K the lens intrinsics, R and t the camera pose, and s a scale factor:

$$s \cdot [u, v, 1]^T = K \cdot [\, r1 \;\; r2 \;\; r3 \;\; t \,] \cdot [X, Y, Z, 1]^T$$

On the table, Z is the same for every point. The term r3 times Z then becomes a constant and merges into t, so one column drops out:

```
s · [u, v, 1]ᵀ = K · [ r1  r2  t' ] · [X, Y, 1]ᵀ  =  G · [X, Y, 1]ᵀ

G is 3 x 3, and t' = t + Z · r3
```

The code uses the inverse, H = G^-1, which goes from pixel to table. The division by w is what makes it a homography and not a plain linear map; it handles perspective:

```
[x', y', w]ᵀ = H · [u, v, 1]ᵀ

X = x' / w
Y = y' / w
```

This is pixel_to_table() in homography_calib.py. The current H, valid for 1280 x 720 images:

```
    | -1.19557e-3   -7.36559e-5    1.09546  |
H = | -9.03705e-5    1.20753e-3   -0.414990 |
    |  9.68483e-6    3.80828e-5    1        |
```

Read it roughly: the top-left 2 x 2 block is the scale (about 1.2 mm per pixel) with the mirror and rotation, the right column is the offset, and the small bottom row is the perspective from the slight camera tilt.

Fitting H from dots (DLT)

H has 9 numbers but only 8 unknowns, because H and 2H give the same result; the code fixes h33 = 1. Each dot gives 2 equations, so 4 dots are the minimum. You used 11, so the fit averages out touch errors. Writing h as the 9 entries of H in a column, each dot (u, v) with table point (X, Y) adds two rows to a matrix A:

```
| -u  -v  -1   0   0   0   X·u  X·v  X |
|  0   0   0  -u  -v  -1   Y·u  Y·v  Y |  · h  =  0

h = the 9 entries of H, row by row
```

With 11 dots, A is 22 x 9. The best h is the right singular vector of A with the smallest singular value (np.linalg.svd). Before that, fit_homography() normalises both point sets: shift them to their centroid and scale them so the mean distance is the square root of 2. Pixels are in the hundreds and metres are below 1, and without this the SVD weights them badly. After the fit, H is un-normalised and divided by h33.

Measuring the dots with the screw tip

For each dot you had the pixel from the camera and the link6 pose from tf2_echo, with a screw clamped in the closed fingers. The screw tip is where the dot is, and it sits a length L below link6 along the tool axis. When the gripper is slightly tilted, the tip moves sideways, so the code corrects for it:

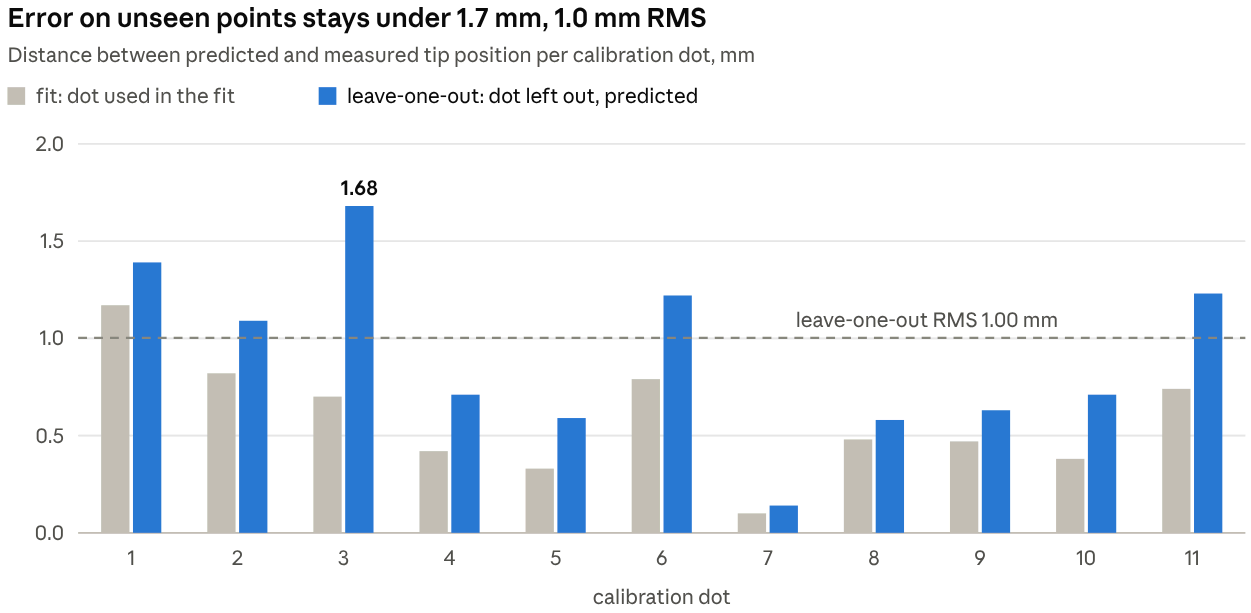
```
p_tip = p_link6 + L · (r13, r23, r33)        L = 0.215 m
```

The column (r13, r23, r33) is the third column of the tf2_echo matrix, the tool axis in base_link. L was found from the data: the fit error is lowest for L between 0.20 and 0.23 m. Without this correction, the dots tilted by 2 deg (dots 8 to 11) were off by 4 to 7 mm. Because the screw sits on the gripper centre and the gripper centre is on the joint 6 axis (your yaw test), the calibrated x, y is also the link6 x, y for a vertical gripper over that point.

Dot	Pixel (u, v)	Tip x, y (mm)
1	522, 401	432.2, 20.8
2	651, 348	286.4, -53.2
3	710, 460.5	207.6, 75.8
4	611, 533.5	317.6, 169.2
5	507, 534.5	439.1, 180.2
6	462, 446.5	499.2, 81.4
7	468, 302.5	505.4, -90.6
8	534, 248.5	432.7, -161.2
9	621, 223	331.3, -198.9
10	685, 185.5	259.0, -249.3
11	504, 158.5	476.7, -265.9

How good it is

Two error numbers matter. The fit error is how far each dot lands from its own measured point. The leave-one-out error is the honest one: fit H on 10 dots, predict the 11th, repeat for each dot. It tells you the error on a block the calibration has never seen.



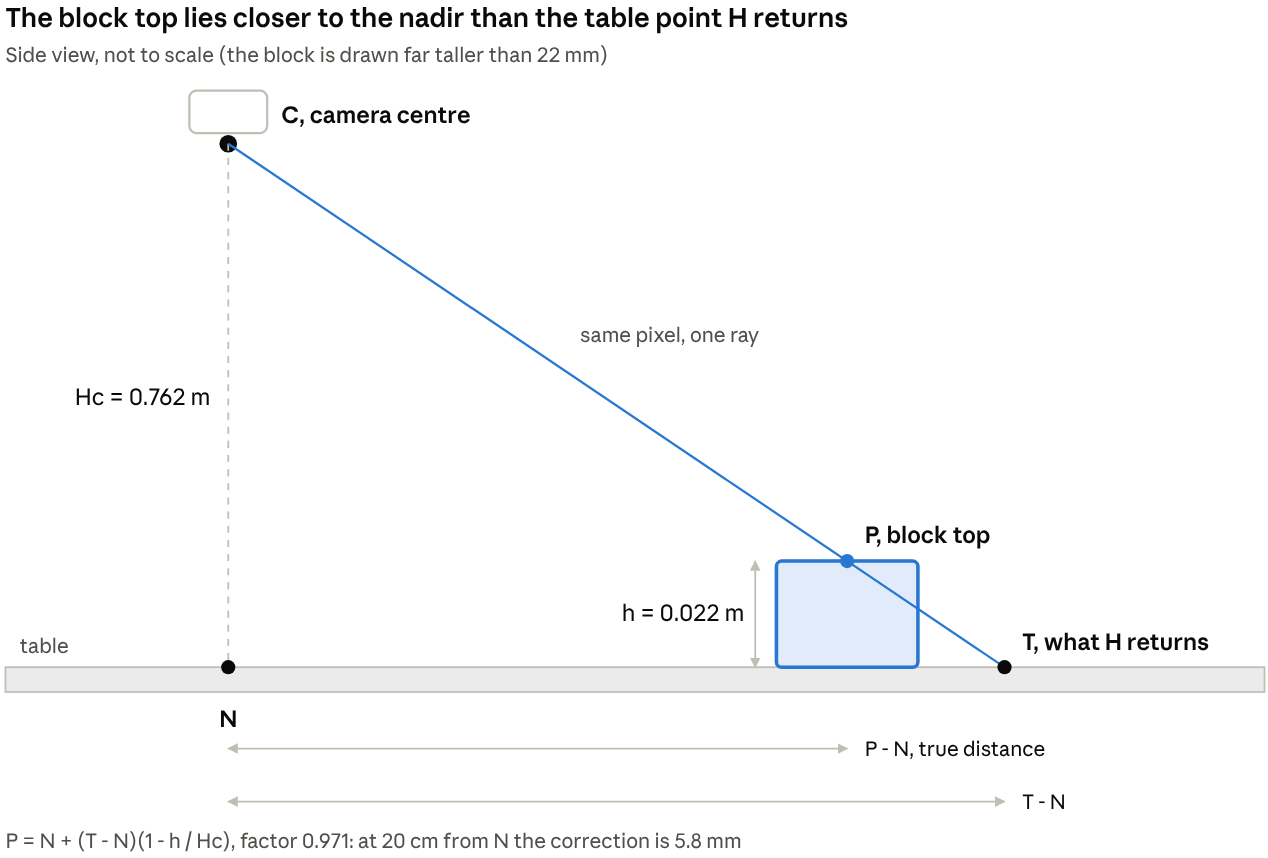
screw-tip calibration, 28 Oct 2026 · 11 dots, tip length 0.215 m

The fit RMS is 0.65 mm. Dot 3, at the image edge, is the worst unseen point. For comparison, the earlier tilted-bolt calibration plus the 3-block offset gave 5.6 mm on unseen points.

The calibrated area is what the dots cover: x 0.20 to 0.51 m and y -0.27 to 0.19 m (CAL_AREA). in_area() allows 2 cm beyond that. Outside it the homography extrapolates, and the error grows.

4. Height correction for the block top

The dots lay on the table, but the camera sees the block's top face, 22 mm higher. Without a correction, every block would be placed too far from the camera's nadir, by 2.9 % of its distance from it: about 5.8 mm at 20 cm.



side view through the camera centre · heights not to scale

The camera centre C is a height Hc above the table, straight above the nadir N. A point on the block top sits at horizontal position P and height h. Its pixel is the same as that of the table point T where the ray from C through the block top hits the table. H gives you T, not P. The triangles C-N-T and the block top give:

```
(T - N) / Hc  =  (P - N) / (Hc - h)

=>  P = N + (T - N) · (1 - h / Hc)
```

This is pixel_to_base(). With h = 0.022 m (19 mm block + 3 mm board) and Hc = 0.762 m, the factor is 0.9711. N = (0.294, -0.0185) m and Hc both come from the camera pose solved from the same 11 dots. The formula is exact for any pinhole camera, tilted or not, because the camera centre, the block top and T are always on one straight line.

The constants live in homography_calib.py: BOARD = 0.003, BLOCK_H = 0.019, BLOCK_TOP_H = 0.022, CAM_HEIGHT = 0.762, NADIR_XY. If you pick without the board, set BOARD = 0; leaving it at 3 mm costs only about 0.8 mm at 20 cm from the nadir.

5. Detection: pixel and angle from YOLO-OBB

The detector gives two things per block: the centre pixel (u, v) and the long-side angle in the image. Nothing else from it is used for the robot.

YOLO-OBB (oriented bounding box) returns, per block: the class, the confidence, xywhr (centre, width, height, rotation) and xyxyxyxy (the 4 corners). The node takes the centre from xywhr. It does not use YOLO's rotation r directly, because YOLO's choice of which side is "width" is not fixed. Instead, long_side_angle() measures the long side itself from the corners c0 to c3:

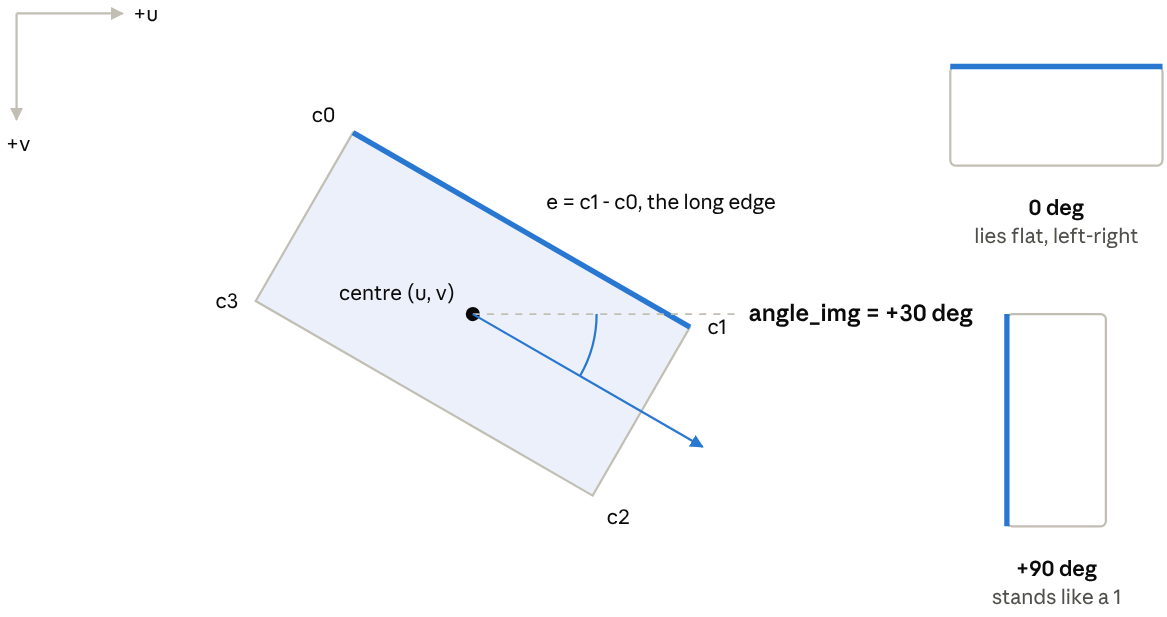
```
e1 = c1 - c0
e2 = c2 - c1
e  = the longer of e1 and e2

angle_img = atan2(e_v, e_u)
```

The angle is then folded into (-90, 90] deg, because a side has no direction: 10 deg and 190 deg are the same line.

The angle is the long side, measured clockwise from image right

One OBB as the detector sees it, in image pixels (v points down)



long_side_angle() on one detection · corners as YOLO returns them

The convention. The angle is measured from +u (right) towards +v (down). On the screen that turns clockwise, because v points down.

- 0 deg: the block lies flat, long side left-right.
- +90 deg: the block stands up-down, like the digit 1.
- +30 deg: the right end is lower than the left end.

What gets published. Every 0.5 s (rate 2 Hz) the node sends one JSON message on /detector/detections: the image stamp and size, and per block its name, conf, u, v, angle_img, long_px, short_px and the corners. Blocks are sorted left to right and numbered; that number is the id shown in the window and used by pick_block.py --id.

One bias to know. The box wraps everything the camera sees of the block, including part of the side faces. Away from the nadir, the camera sees the outer side face, so the box centre moves outward by roughly half of that face's width: about 3 mm at 20 cm from the nadir.

6. From image angle to a direction on the table

The image angle cannot simply be copied to the robot, because the image is rotated, mirrored and slightly tilted against the table. The code maps the direction through the same homography instead, so it stays correct anywhere in the image.

Two-point method (image_angle_to_base_yaw). Take the block centre and a second point 20 px along the long side in the image. Map both to the table at block-top height, and measure the direction between them:

```
P0 = f(u, v)
P1 = f(u + 20 · cos(angle_img),  v + 20 · sin(angle_img))

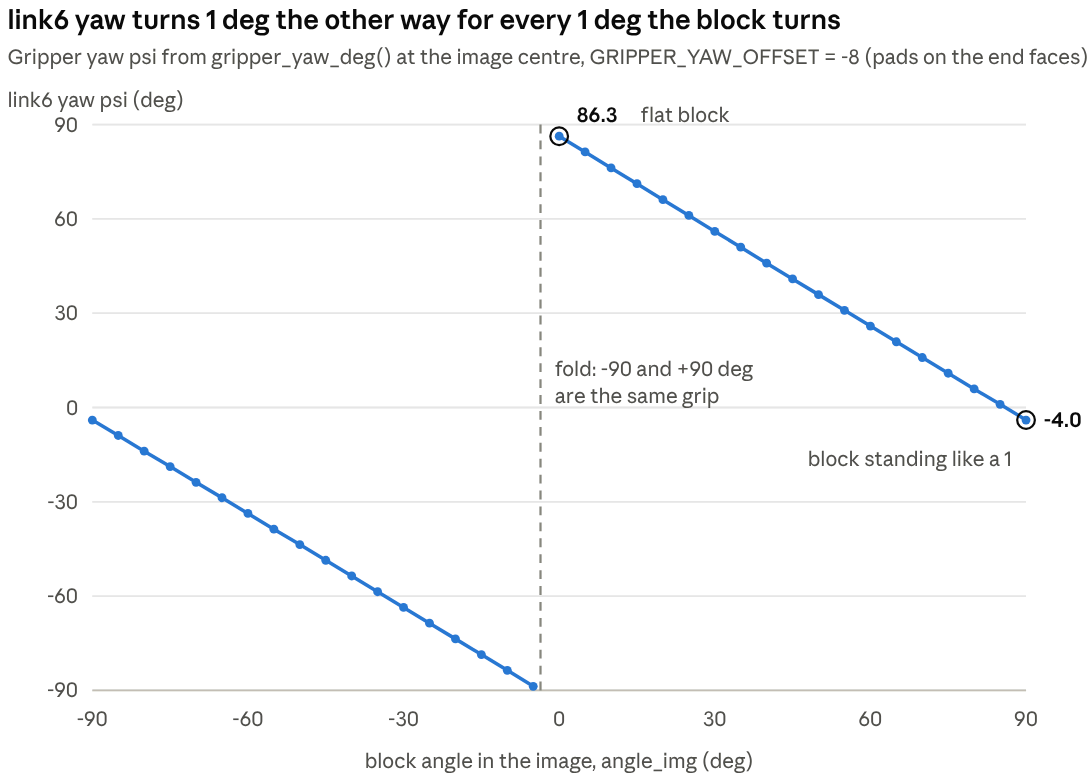
theta_long = atan2(P1.y - P0.y,  P1.x - P0.x)
```

Here f is pixel_to_base(), the homography plus the height correction. The result is the direction of the block's long axis in base_link, measured from +x towards +y.

What it works out to in your setup. Near the image centre, the mapping is almost exactly:

```
theta_long  ≈  -175.7 deg  -  angle_img
```

The minus sign in front of the image angle is the mirror from section 2: a block turned clockwise on the screen is turned counter-clockwise on the table. The -175.7 deg is the direction that image right (+u) points on the table. Two checks: a flat block (0 deg) lies along about -x of the robot, and a block standing like a 1 (90 deg) lies along about +y.



computed with homography_calib.py at pixel (640, 360) · image angle every 5 deg

The slope is -1 because of the mirror. The jump at about -3.6 deg is not a real jump on the robot: at that block angle, link6 switches from -90 deg to +90 deg, which grips the block the same way.

Folding. A long axis is a line, so -175.7 deg and +4.3 deg describe the same block. fold90() brings any angle into (-90, 90] deg by adding or removing 180 deg. Section 7 uses it on the gripper yaw.

7. Gripper yaw and the quaternion

The gripper yaw psi is the one angle the robot needs besides x, y, z: it is the direction of link6's x axis on the table, the same number as the yaw in tf2_echo base_link link6. It is not a joint 6 angle; MoveIt finds the joints.

link6 pointing straight down

A vertical gripper turned by psi is a 180 deg turn about x (flip the tool down), then a turn psi about the vertical:

```
                          |  cos psi    sin psi    0 |
R = Rz(psi) · Rx(180) =   |  sin psi   -cos psi    0 |
                          |    0          0       -1 |
```

As quaternions (x, y, z, w), q_z(psi) = (0, 0, sin(psi/2), cos(psi/2)) and q_x(180 deg) = (1, 0, 0, 0). Their product is:

```
q = q_z(psi) ⊗ q_x(180) = ( cos(psi/2),  sin(psi/2),  0,  0 )      order (x, y, z, w)
```

That is down_quat() in goto_block.py. Check with your own reading: tf2_echo showed yaw 30.5 deg and quaternion (0.965, 0.263, 0.001, 0.001), and cos 15.25 deg = 0.965, sin 15.25 deg = 0.263. In RPY that pose reads (180, 0, psi).

Which way the fingers close

The fingers do not close along link6's x axis. You measured it: with the pads squeezing the long faces of the red block, link6 was at -4.15 deg, while that block's long axis was at -175.6 deg. The pads then closed across the long axis, at -85.6 deg, the same line as +94.4 deg. So the closing axis sits about beta = 98 deg from link6 x (94.4 + 4.15 = 98.5). The 8 deg beyond a right angle is how the gripper sits on the flange, plus a little detector and eye error.

The yaw formula in the code

```
psi = fold90( theta_long - 90 deg + delta )        delta = GRIPPER_YAW_OFFSET
```

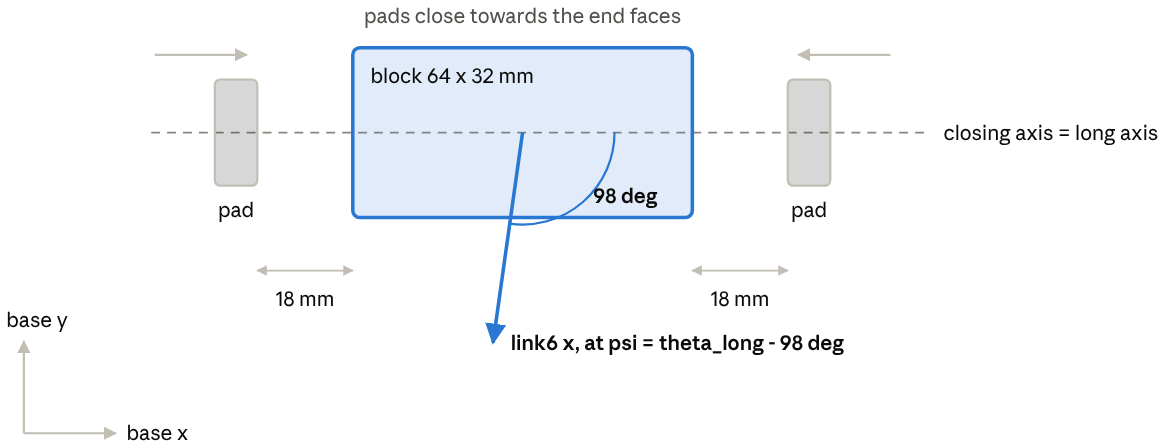
The fingers then close along psi + 98 deg. Fill in the two settings:

delta	Closing axis	Pads touch	Squeezes	Clearance per side
-8 (current)	Along the long axis	The two 32 mm end faces	The 64 mm length	(100 - 64) / 2 = 18 mm
+82	Across the long axis	The two 64 mm long faces	The 32 mm width	(100 - 32) / 2 = 34 mm

With delta = -8, psi = theta_long - 98 deg, and the closing axis comes out as psi + 98 = theta_long: exactly along the block.

With offset -8 the pads close along the block and touch the end faces

Top view in base_link, block long axis drawn along +x; gripper open (100 mm), to scale



gripper over a block, top view · offset -8, beta = 98 deg measured on 28 Oct

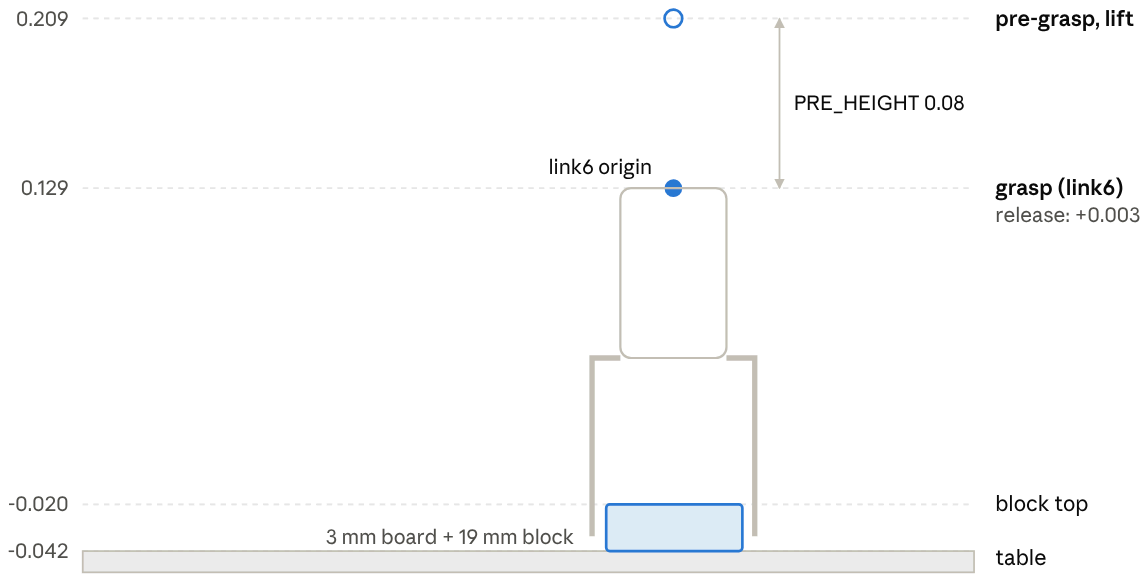
Why fold. A two-finger gripper looks the same after a half turn, so psi and psi + 180 deg give the same grip. fold90() keeps psi in (-90, 90] deg, so link6 never has to turn more than 90 deg away from its psi = 0 pose. That keeps joint 6 away from its limits and shortens the moves. A block standing like a 1 needs about -4 deg, and a flat block about +86 deg.

8. Heights, motion and the pick and place sequences

x, y and yaw come from the camera; z comes from fixed, measured heights. All z values are link6 heights in base_link, about 15 cm above the fingertips.

link6 stops at z 0.129 m to grasp, and 8 cm higher before and after

Side view, heights to scale, z in base_link (m); the flange sits about 16 cm above the fingertips



heights used by goto_block.py · GRASP_Z, PRE_HEIGHT and the table and block heights

Height	Value (m)	Where it comes from
Table top	-0.042	TABLE_Z, from the screw-tip dots
Block top	-0.020	Table + 3 mm board + 19 mm block
Grasp (link6)	0.129	GRASP_Z, measured: fingers closed on a block, tips 3 to 5 mm above the board
Pre-grasp and lift	0.209	Grasp + PRE_HEIGHT 0.08
Release at the place	0.132	Grasp + 0.003, so the block is set down, not pressed

One move (Robot.move)

Each move is one goal sent to MoveIt's /move_action (move_group) for group igus_rebel_arm:

- Position constraint: link6's origin inside a sphere of 1 mm radius around (x, y, z).

- Orientation constraint: link6 within 0.01 rad (0.57 deg) of down_quat(psi) about each axis.
- Planner: OMPL (RRTConnect in your config), up to 5 attempts in 5 s, then executed.
- Speed: velocity and acceleration scaled to 0.05 by default, never above 0.2.

MoveIt plans in joint space. The start and end poses are exact, but the path between them is not forced to be a straight line. A descent from pre-grasp to grasp is normally close to straight because it is short, but nothing guarantees it.

The gripper (Robot.gripper)

Two digital outputs drive it through the service /set_digital_output: output 30 closes, output 31 opens. To open, the code switches 30 off, switches 31 on, waits GRIP_TIME (3 s) and switches 31 off again. Closing is the same with the outputs swapped. Switching the other output off first means open and close are never on together.

pick_block.py from start to end

1. **Read one detection.** It subscribes to /detector/detections and takes the first message that arrives at least 0.3 s after it started, so it never uses an old picture. The arm must be out of the camera view at this moment.
2. **Convert every block.** Blocks under --min-conf (0.5) are dropped. For each of the rest it computes x, y (section 4) and psi (section 7) and prints the table.
3. **Choose.** --color takes the most confident block of that colour, --id takes that number, and without either it asks you.
4. **Safety checks.** The target must be inside the calibrated area (in_area). With --mode place, the place spot is converted the same way from its pixel (--place-u, --place-v) and angle (--place-angle), or from a link6 yaw given directly (--place-yaw), and it must also be inside the area unless you add --allow-outside.
5. **Pick** (pick_sequence): pre-grasp, open, down to grasp z, close, up to pre-grasp.
6. **Place** (place_sequence, only with --mode place): above the place spot at its yaw, down to release z, open, up.

With --step pause (the default) it waits for Enter before each step. With --step direct it runs straight through. If any move fails, it stops and leaves the robot where it is.

9. Worked example: the red block

The red block from your yaw test, at pixel (495, 529) with image angle 0 deg, becomes link6 x 0.4488 m, y 0.1691 m and yaw 86.4 deg. Every number below can be checked with python3 homography_calib.py query 495 529.

1. **Pixel to table.** H times (495, 529, 1), divided by w, gives T = (0.4534, 0.1747) m. That is where this pixel's ray meets the table.
2. **Height correction.** P = N + (T - N) x 0.9711 with N = (0.294, -0.0185) gives P = (0.4488, 0.1691) m. The block top is 4.6 mm closer in x and 5.6 mm closer in y to the nadir than T.
3. **Long axis.** The second point is 20 px to the right, pixel (515, 529), which maps to (0.4260, 0.1674) m. The direction from P to it is atan2(-0.0017, -0.0228) = -175.6 deg: the block lies almost along -x.
4. **Gripper yaw.** psi = fold90(-175.6 - 90 - 8) = fold90(-273.6) = 86.4 deg.
5. **Quaternion.** (cos 43.2 deg, sin 43.2 deg, 0, 0) = (0.729, 0.684, 0, 0).
6. **Fingers.** They close along psi + 98 = 184.4 deg, the same line as -175.6 deg: along the block, pads on the two end faces.
7. **Moves.** Pre-grasp (0.4488, 0.1691, 0.209), open, grasp (0.4488, 0.1691, 0.129), close, lift back to 0.209.

A cross-check from your measurement: with the pads on the long faces this block needed link6 at -4.15 deg. The current setting turns that by 90 deg, to 86 deg, which is the end-face grip.

10. Error budget, limits and when to recalibrate

The calibration itself is the smallest error, about 1 mm. The detector's box centre and the side-face bias add a few mm, which the end-face grip absorbs with its 18 mm of clearance per side along the block.

Source	Size	Note
Homography on unseen points	1.0 mm RMS, worst 1.7 mm	Leave-one-out over 11 dots
Box centre from YOLO	1 to 2 px, so 1.2 to 2.4 mm	Varies from frame to frame
Box includes side faces	About 3 mm, pointing away from the nadir	Grows with distance from the image centre

Source	Size	Note
Detector angle	About +-4 deg seen (-3.5 to +4 on one block)	At 32 mm from the centre, 4 deg moves a pad by about 2 mm
Board height left wrong	About 0.8 mm at 20 cm from the nadir	Only if BOARD does not match reality
Gripper tilt	About 3.5 mm per degree at the screw tip	MoveIt holds 0.57 deg; it mattered only for hand-jogged dots

In your yaw test, the closed gripper sat 9 mm from the homography position. About 3 to 4 mm of that came from a 1.5 deg tilt in the hand jog, and most of the rest fits the side-face bias.

Limits

- The default place spot, pixel (577, 104), maps to x 0.391, y -0.329 m. That is outside the calibrated area (y down to -0.29 m with the margin), so place mode refuses it unless you add --allow-outside. Add 2 or 3 dots near the place spot and refit, or move the spot inside the area.
- Only flat objects at a known height work. A stacked block or another object height needs its own h in pixel_to_base().
- The descent is planned in joint space, so it is not guaranteed to be straight.
- The arm must be out of view while the detection is read.

Recalibrate (redo the dots, then homography_calib.py fit) when

- the camera, its stand or its mount moved, even slightly;
- the image size is no longer 1280 x 720;
- the robot base or the table moved.

Change only the constants (no new dots) when the board or table height changes, or when you change the gripper fingers (measure GRASP_Z and GRIPPER_YAW_OFFSET again).

A quick health check any time: python3 homography_calib.py --selftest maps the 11 dots back and should report a mean of about 0.6 mm.